

Trabajo Práctico Grupal

Materia: Algoritmos y Estructura de Datos

Profesora: Yamila Ariadna Zakhem

Integrantes:

- Andrea Dentice
- Micaela Aviza
- Noelia Escobar
- Maia Velazquez

Curso: K1151

Grupo: N° 2

Año lectivo: 2026

1) Enunciado original que desarrollamos

Una veterinaria almacena información de sus pacientes en un archivo binario denominado “pacientes.dat”. De cada paciente se registra el nombre, la edad, el estado de salud (‘S’ si se encuentra sano o ‘E’ si se encuentra enfermo) y su historial de vacunación.

Cada paciente puede tener registrada una cantidad variable de vacunas. De cada vacuna se almacena el nombre, la fecha de aplicación (en formato AAAAMMDD) y su estado (‘S’ si fue aplicada o ‘P’ si está pendiente).

a. Generar una lista de pacientes que tengan al menos una vacuna pendiente.

Considerando que la veterinaria tiene distintos tipos de vacunas. Una limitante es que solo la vacuna “eutanasia” puede ser aplicada como máximo tres veces por día. Cuando se acerca un paciente indica su motivo de consulta.

A medida que ingresan pacientes, son gestionados mediante una cola para su atención. Si el motivo de consulta es “eutanasia” ir descontando de la cantidad máxima. Si se pasa la cantidad máxima, no ingresar el paciente a la cola.

b. Se pide hacer el control de pacientes mediante una cola.
c. Mostrar por pantalla la cantidad de pacientes atendidos en el día.

2) Enunciado que recibimos para resolver (del grupo N° 5)

Se tiene un vector de jugadores de fútbol que posee los siguientes campos de información: Nombre, Edad, Goles y Posición (P: portero, D:defensor, M:mediocampista, A:atacante), en un archivo llamado “PLANTEL.dat”. El archivo está desordenado y se deben realizar los siguientes casos:

a. Ordenar el archivo de la siguiente manera: primero por posición (empezando por el portero, luego el defensor, después el mediocampista y por último el atacante), y por edad.
b. Generar un vector para mostrar por pantalla los jugadores cuya edad sea menor de 23 años y qué posición del campo ocupan. Se sabe que hay, como máximo, 26 jugadores
c. Crear una nueva pila con el nombre de los 5 máximos goleadores, incluyendo la cantidad de goles que hizo.

3) Detalle de las modificaciones que hicimos en el enunciado recibido

Para identificar a los jugadores en vez de nombre, utilizamos código de jugador. Para las posiciones, en lugar de ser tipo de dato char, las representamos con un determinado número (1 para el portero, 2 para defensor, 3 para mediocampista y 4 para atacante).

4) Estructuras necesarias para resolver el enunciado recibido

```
struct Jugador {
    int codigo;
    int edad;
    int goles;
    int posicion;
};
struct Nodo {
    Jugador info;
    Nodo* sgte;
};

Jugador menoresA23[26];

Nodo* lista = NULL;

Nodo* pila = NULL;
```

5) Estrategia de resolución

Primero, definimos nuestras estructuras:

- Jugador: almacena los datos de cada jugador (código, edad, goles y posición).
- Nodo: almacena un jugador y un puntero al siguiente nodo.
- Vector menoresA23: almacena los jugadores menores de 23 años. Tiene capacidad para 26 jugadores porque esa es la cantidad máxima indicada.
- Lista: almacena a los jugadores ordenados por posición y, dentro de cada posición, por edad.
- Pila: permite almacenar temporalmente a los cinco máximos goleadores.

A medida que vamos leyendo, evaluamos si el jugador tiene una edad menor a 23. Si cumple la condición, se guarda en el vector menoresA23. También cada jugador que vamos leyendo lo vamos insertando ordenadamente en una lista respetando

primero por posición y luego por edad, ambas de menor a mayor. Este procedimiento se repite hasta terminar de leer el archivo.

Una vez leídos todos los jugadores, volvemos a abrir el archivo pero esta vez en modo escritura. Creamos un puntero aux que (valga la redundancia) apunta al primer nodo de la lista con el fin de que mientras se recorre aux se va escribiendo el archivo ordenadamente.

Una vez el archivo está reescrito, mostramos por pantalla los jugadores del vector menoresA23 y el top de goleadores.

Para almacenar el top de goleadores en una pila utilizamos todos los jugadores guardados en la lista. A medida que recorre la lista, busca el máximo goleador. Una vez lo encuentra, se lo pushea a la pila y se lo elimina de la lista. Repetimos esto hasta obtener los cinco máximos goleadores o hasta que la lista quede vacía. Si no hay suficientes jugadores para completar el Top 5, mostramos un mensaje indicándolo. Finalmente, mostramos los goleadores almacenados en la pila.

6) Código que resuelve el problema

Link del main.cpp